

令和5年

プロジェクトデザインⅢプロジェクトレポート

カメラ画像を用いたマルチコプタ によるライントレース

指導教員

伊藤恒平

金沢工業大学

工学部ロボティクス学科

4ER2-47 水谷真宙

目次

第 1 章	はじめに	3
1.1	研究の背景	3
1.2	研究の目的	4
第 2 章	ライントレースによる機体制御	5
2.1	機体について	5
2.2	マルチコプタの飛行システム	6
第 3 章	カメラ画像を用いたラインとの距離検出	8
3.1	画像座標系から慣性座標系への座標変換	8
3.2	ラインとの距離検出実験	13
3.3	実機での模擬飛行	14
3.4	ラインとの距離検出実験に対する考察	15
第 4 章	Simulink による線形・非線形のライントレースシミュレーション	17
4.1	線形シミュレーション	17
4.2	非線形シミュレーション	22
4.3	線形シミュレーションと非線形シミュレーションの結果 による考察	29
第 5 章	おわりに	31
5.1	結論	31
5.2	今後の課題	31

参 考 文 献	32
謝 辞	33

第1章

はじめに

1.1 研究の背景

近年安定した飛行を行うことができるドローンの開発が発展しており、小型ドローンの普及も進んできている。マルチコプタは、複数のプロペラを使用することで、安定した飛行を実現している。土木現場の測量・三次元化、土木進捗、農地のリモートセンシングなど様々な用途で使用されており、市場規模がだんだん大きくなってきている。市場規模が大きくなっていくことを示す図を図1.1に示す。図1.1に示したように、産業活動での期待が高まっている [1]。

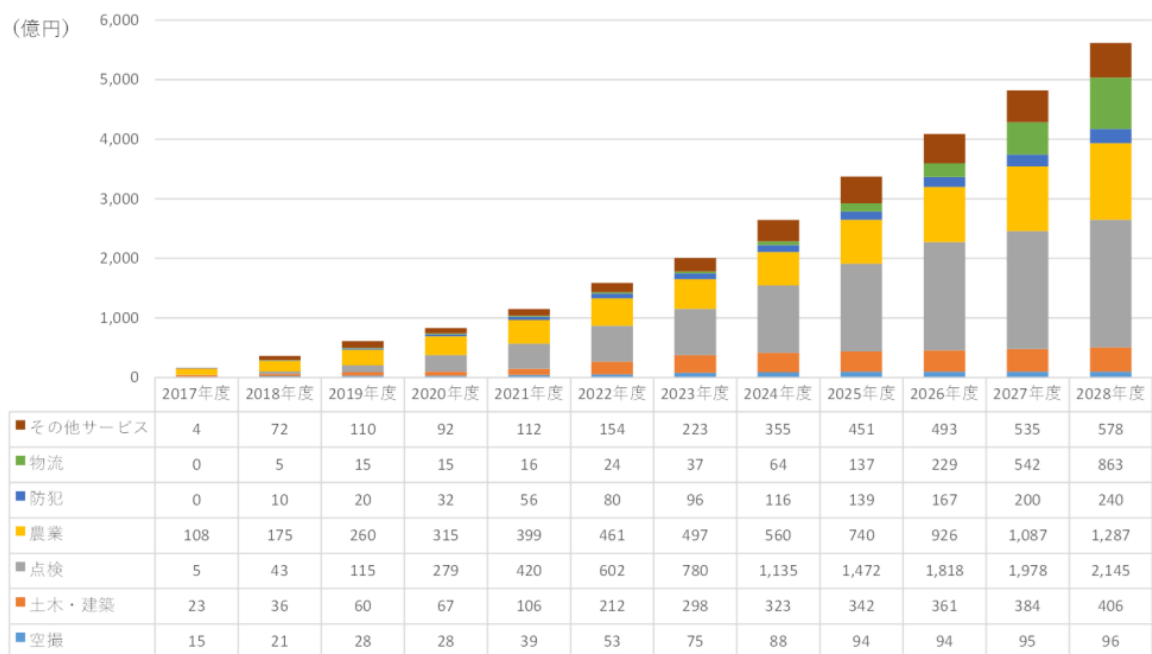


図 1.1 国内ドローンビジネスのサービス別市場規模

出典: インプレス総合研究所作成

マルチコプタを手動で操縦する際、機体を視認できない場合、もしくは視認しにくい場合、操縦者はマルチコプタの姿勢や位置が把握できず操縦が難しくなってしまう。そのような場合に一定の経路の連続的な飛行や自動的な帰還などを行うために、何らかの指標を基にした自律飛行能力が必要となる [2]。

現在の自律飛行は GPS で自らの位置を把握しながら指定された場所に移動させる手段が一般的である。しかし、全日本学生飛行ロボットコンテストのような GPS の届かない室内でかつ、重量制限などのルールにより多くのセンサを使用できない状況で高度な制御をできるかを競い合う大会 [3] などもある。GPS が届かない・届きにくい場所でも安定した自律飛行が求められている。

1.2 研究の目的

本研究では、屋内での飛行と仮定して、GPS を用いないドローンの誘導方式として赤色 LED のラインを使用することを考える。赤色 LED ラインを検出するためにカメラ画像を用いて行う。カメラは機体に固定されている。カメラ画像情報よりラインとの距離を検出するように考えた。その際の問題点として、カメラ画像情報だけでは正確なラインとの距離を検出することができない点である。カメラによるラインの見え方だけでは機体とラインとの距離を得ようとしたとき、機体がライン上にあったとしても、機体の姿勢によってカメラによる見え方が変わってしまい、正確なラインとの距離を検出することができない。そこで、カメラ画像上のラインの位置を慣性空間上に座標変換することで機体自身の位置を推定することができ、カメラ画像上のラインの見え方によってラインと機体との距離が変わってしまうことをなくす。本研究では、カメラ画像を用いて正確なラインとの距離を検出し、マルチコプタをライントレースすることが目的である。

第 2 章

ライントレースによる機体制御

2.1 機体について

本研究では,OpenMv カメラと Raspberry pi pico を用いて,画像認識から飛行制御を行う. 本研究で使用する機体に取り付けられているセンサは,カメラ,ジャイロセンサ,地磁気センサ,高度センサである。そのため本研究で使用する機体のセンサから読み取れる物理量は,角速度 (p,q,r) , マルチコプタの姿勢を表すクォータニオン (q_0,q_1,q_2,q_3) , ラインである. このようなものから推定することができる値は,角度 (Φ, Θ, Ψ) である. そして,カメラ画像から読み取れるラインの位置と高度センサから推定できる自己位置 (Y,Z) である. 本研究で使用するマルチコプタの機体を図 2.1 に示す.

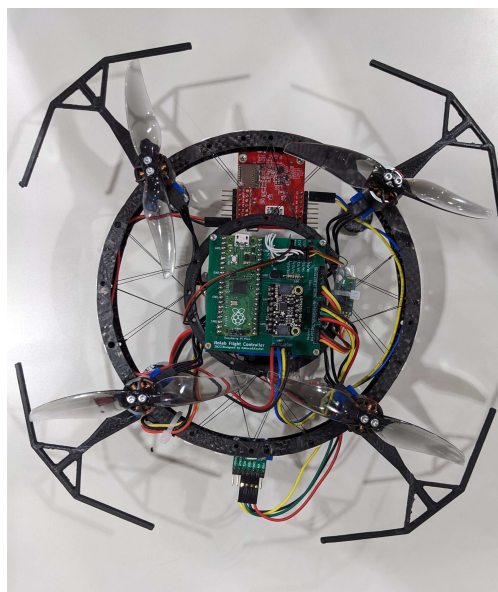


図 2.1 使用したマルチコプタ

2.2 マルチコプタの飛行システム

本研究は以下の図(2.2)に示した手順を踏んでマルチコプタのライントレースを実現させる。

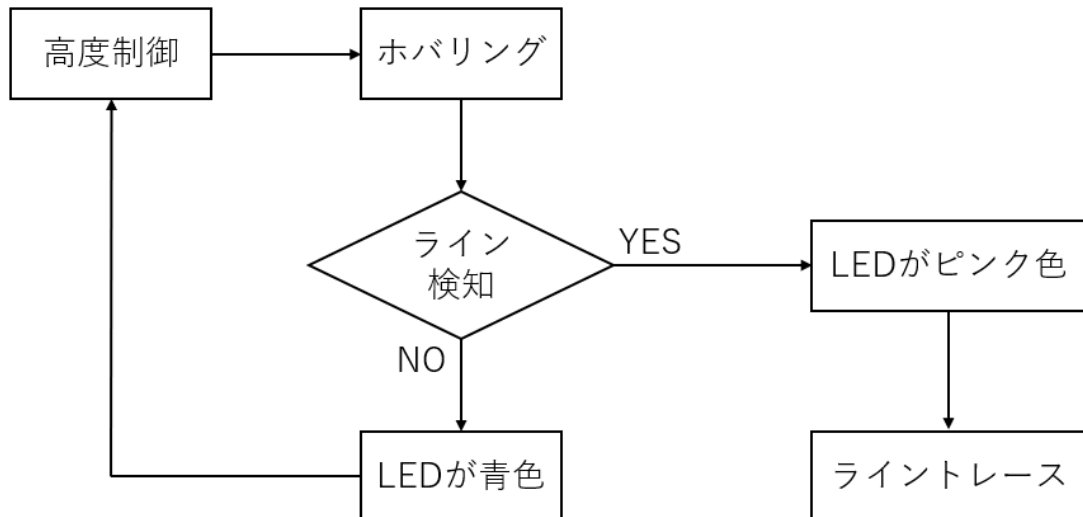


図 2.2 飛行システムの概要

また, 本研究の機体にはモード変更できる仕様をつけており, これらはコントローラについているスイッチでモード切替を可能にしている. 手で操縦するノーマルモードと完全自律飛行でのライントレースモード切替の機能を備えている. ただし, ライントレースモードの際, ライントレースできず, ラインからそれてしまいものにぶつかったり, 人を傷付けてしまうなどの事故を防止するため, 操縦者はライントレースモードであっても, コントローラを持ち, ライントレースモードからノーマルモードにスイッチで切り替えれるようにしておく必要がある. また, 本研究で使用する機体のフレームには LED ライトが前後左右の4箇所につけられており, ライントレースモードになると, カメラがライン検知をしているときは, LED がピンク色に光る. カメラがラインを検知していないときは, LED が青色に光るといった仕様になっている. 上記の図(2.2)の飛行を実現させるために, ラインを認識するカメラ, 認識しているラインの画像を処理する OpenMV, 画像処理の結果を座標変換し, マルチコプタへ制御入力を行う Raspberry pi pico が必要となる. システム全体の

構成を以下の図 (2.3) に示す.

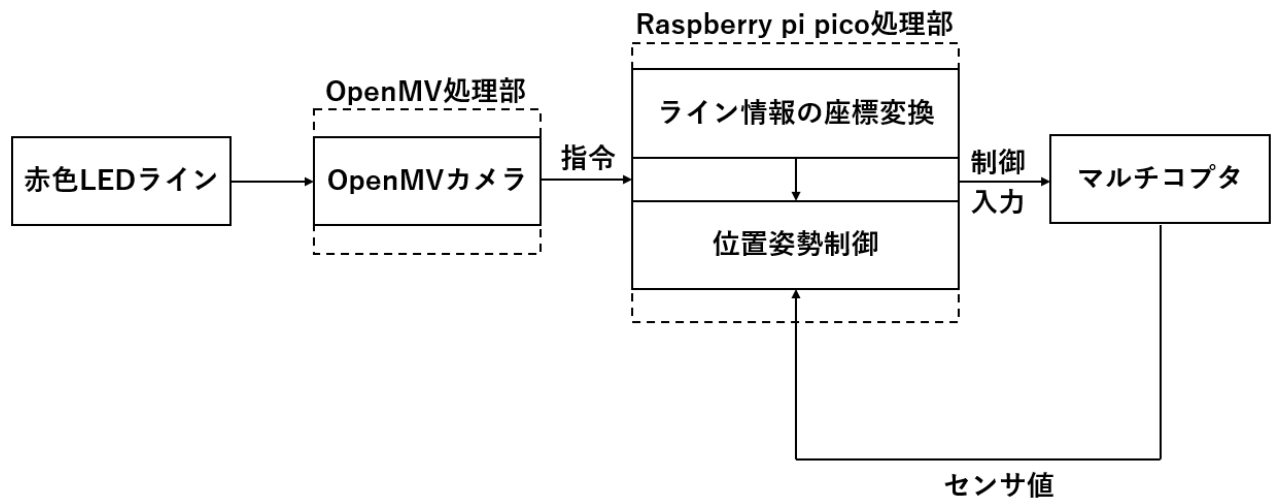


図 2.3 システム全体の構成図

OpenMV によって行われた画像処理によるラインの2点の xy 座標の情報を Raspberry pi pico に送信する際は float 型のバイト列に変換して通信を行う.

第 3 章

カメラ画像を用いたラインとの距離検出

3.1 画像座標系から慣性座標系への座標変換

室内に設けた赤色 LED のラインをカメラで検知し, 検知したラインをカメラ画像座標空間から地球上の座標系の慣性座標空間に, 機体も機体座標空間から慣性座標空間にそれぞれ座標変換することにより位置のずれを算出し, ラインとの距離を検出する. 慣性座標空間のイメージとして図 3.1 を示す. 図 3.1 の X 軸はラインに一致する. また, 慣性空間でのラインに関する座標を (X_L^I, Y_L^I, Z_L^I) とし, マルチコプタ (機体) に関する座標を (X_d^I, Y_d^I, Z_d^I) とおく. マルチコプタとラインとの距離を d とおく.

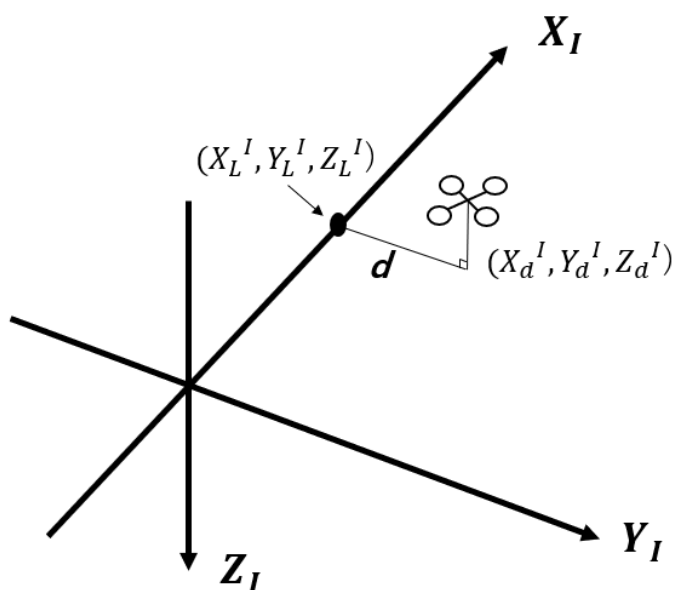


図 3.1 慣性座標空間のイメージ図

図 3.1 より, $d = Y_d^I$ となる.

本研究で使用したカメラは,OpenMV-H7-plus である.OpenMV の画像座標とピクセルのイメージ図を図 3.2 に示す. 画像座標系でのある座標を (x_p, y_p) とすると, 座標の表し方は $(\Delta x \cdot x_p + \Delta x/2, \Delta y \cdot y_p + \Delta y/2)$ と表すことができる. $\Delta x, \Delta y$ は画素の大きさ、 H_p は横の画素数, V_p は縦の画素数, W, H はイメージセンサの x, y の長さである.

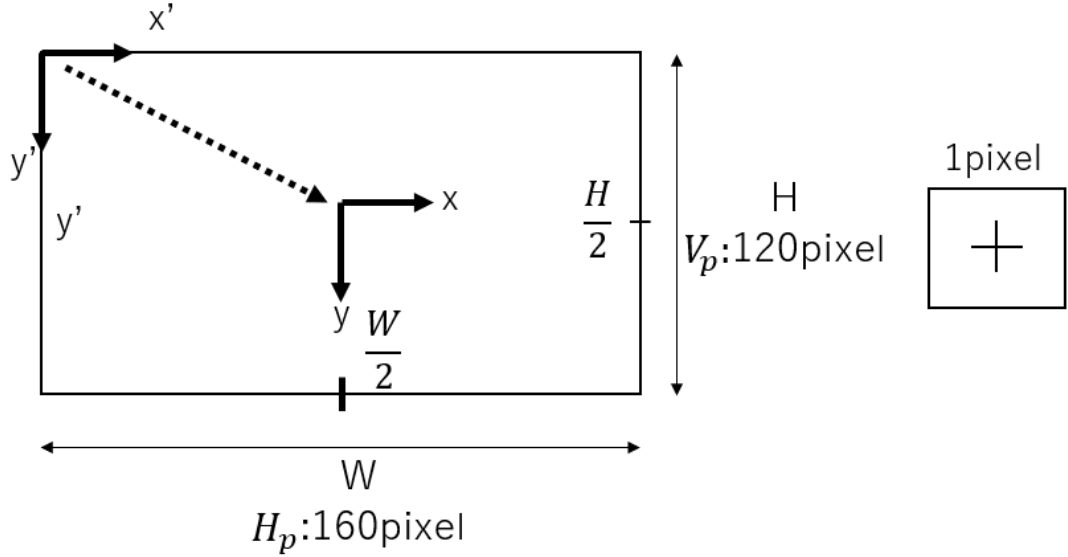


図 3.2 OpenMV の画像座標

図 3.2 に示したように, ラインが機体の中心からどれくらい離れているかの情報を得たいため, $x' \rightarrow y'$ 座標を画像の中心の $x \rightarrow y$ 座標へ平行移動する必要がある. 平行移動の式はラインの画像座標を (X_L^M, Y_L^M) とすると、式 (3.1),(3.2) の様になる.

$$X_L^M = \Delta x \cdot X_L + \Delta x/2 - \Delta x \cdot H_p \quad (3.1)$$

$$Y_L^M = \Delta y \cdot Y_L + \Delta y/2 - \Delta y \cdot V_p \quad (3.2)$$

式 (3.1),(3.2) を使用することで, 画像座標系の原点の初期設定を行うことができる. 式 (3.1),(3.2) 中に使用している画素の大きさである $(\Delta x, \Delta y)$ の求め方について説明する. その際, ピンホールカメラモデルを図示した図 3.3 を使用して説明する.

図 3.3 に示した x' はカメラから得られるカメラ座標の x 座標, f_x は本研究で使用したカメラの焦点距離, H は地面との距離, X はラインと機体と

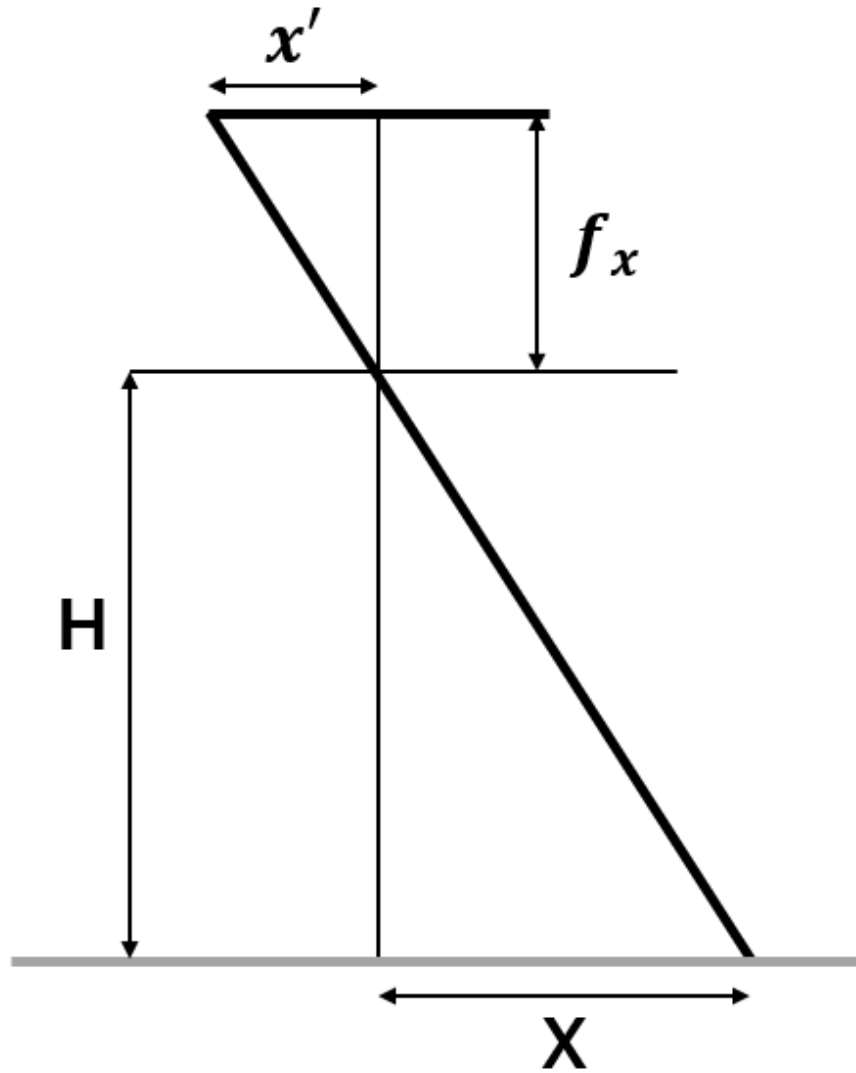


図 3.3 ピンホールカメラモデル

の距離である. カメラをどの回転軸にも回転させず, 地面と水平状態のままラインとカメラとの距離 X が 100[mm] となる地点で, 地面との高さ H が 500[mm] のとき, カメラがラインを認識する座標はどこか調べた. このような条件でデータを取得したところ, カメラが認識する座標の X 座標は 48[pixel] となった. カメラ座標系の中心となる X 座標は 80[pixel] であるため, カメラの中心とカメラが認識したラインの X 座標の差は 32[pixel] となり, カメラ座標系の中心とカメラ座標系のラインの位置の距離 x' が 32[pixel] となる. この条件のもとで画素の大きさ Δx を求める. 画素の大きさを求める式を式 (3.3) に示す.

$$\begin{aligned}
x'/H &= f_x/H \\
X &= H/f_x \cdot \Delta x \cdot x' \\
\Delta x &= X \cdot f_x/x' \cdot H
\end{aligned} \tag{3.3}$$

したがって, 上記の式に焦点距離の値, 高さの値, ラインと機体との距離を代入することで画素の大きさを求めることができる. 機体をどの回転軸にも回転させず, 地面と水平状態のままでラインと機体との距離 X が 100[mm] となる地点で, 地面との高さ H が 500[mm], カメラ座標系の中心とカメラ座標系のラインの位置の距離 x' が 32 の条件を式 (3.3) に代入すると, 画素の大きさが 0.0175[mm] であると分かった.

次に, 画像座標からカメラ座標へ変換する. このとき, ピンホールカメラモデルの透視投影変換行列の内部パラメータを用いて考える. 透視投影変換行列の内部パラメータは式 (3.4) のようになる. カメラ座標系のラインに関する座標を (X_L^C, Y_L^C, Z_L^C) とすると,

$$\begin{bmatrix} X_L^C \\ Y_L^C \\ Z_L^C \end{bmatrix} = s \begin{bmatrix} 1/f_x & 0 & 0 \\ 0 & 1/f_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_L^M \\ Y_L^M \\ 1 \end{bmatrix} \tag{3.4}$$

s は高度センサから得られる地面との距離である. また, f_x, f_y は透視投影変換における逆変換のパラメータである.

次にカメラ座標系から機体座標系に座標変換する. 機体座標系は慣性座標空間と同じく, 右手系で考える. 図 3.1 に示すように, 機体の前進方向を X 軸とする. カメラ座標系は図 3.2 の様な座標系である. そのため, カメラ座標系と機体座標系を比較すると, X^C は Y^B に相当し, Y^C は X^B に相当するため, 次のような式が成り立つ.

$$\begin{bmatrix} X_L^B \\ Y_L^B \\ Z_L^B \end{bmatrix} = \begin{bmatrix} -Y_L^C \\ X_L^C \\ Z_L^C \end{bmatrix} \tag{3.5}$$

式 (3.5) によって, カメラ座標系から機体座標系へ変換することができる. 次に機体座標空間から慣性座標空間への座標変換を考える. この時,

クォータニオンを用いた座標変換行列 E を用いて考える. クォータニオンを用いた座標変換行列 E は式 (4.13) となる. クォータニオンは機体に姿勢を表している.

$$E = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & q_0^2 - q_1^2 - q_2^2 - q_3^2 \end{bmatrix} \quad (3.6)$$

座標変換行列 E より, 機体座標系から慣性座標系に変換する式は式 (3.7) の様になる.

$$\begin{bmatrix} X_L^I \\ Y_L^I \\ Z_L^I \end{bmatrix} = E \begin{bmatrix} X_L^B \\ Y_L^B \\ Z_L^B \end{bmatrix} + \begin{bmatrix} X_d^I \\ Y_d^I \\ Z_d^I \end{bmatrix} \quad (3.7)$$

このとき, 慣性座標空間での X 軸は赤ラインと一致しているため, Y 方向は X 軸を基準としてラインとの距離を得るため $Y_L^I = 0$ である. また, 赤ラインは地面に設けるため $Z_L^I = 0$ となる. 最終的に求めたいのは機体とラインがどれだけ離れているかなので, X^I は未知数のままで考え, 機体自身の位置座標を求めるように式 (3.7) を変形したものを式 (3.8) に示す.

$$\begin{bmatrix} X_d^I \\ Y_d^I \\ Z_d^I \end{bmatrix} = \begin{bmatrix} X_L^I \\ 0 \\ 0 \end{bmatrix} - E \begin{bmatrix} X_L^B \\ Y_L^B \\ Z_L^B \end{bmatrix} \quad (3.8)$$

以上の順でカメラの画像座標系から慣性座標系への座標変換を行うような処理を実装させた. これらの処理によって, 機体とラインを同じ慣性座標空間に置くことができ, ラインと機体との距離 d を算出することができる. 求めたいラインと機体との距離 d は慣性座標空間上では Y 方向のことである. したがって, 式 (3.8) の Y に関する式に注目すると式 (3.9) になる.

$$Y_d^I = -E \cdot \begin{bmatrix} X_L^B \\ Y_L^B \\ Z_L^B \end{bmatrix} \quad (3.9)$$

図 3.1 より, Y_d^I は d に相当するので, 式 (3.9) によって, 慣性座標空間上でのラインと機体との距離を算出することができた.

3.2 ラインとの距離検出実験

まず, 機体は地面に対して水平状態で, 機体を地面から約 1[m] 離れた高さで Y 方向に $\pm 100[\text{mm}]$ と離す. その際に離れた分の距離を正確に認識するかを確かめた. 計測したデータを図 3.4 に示す.

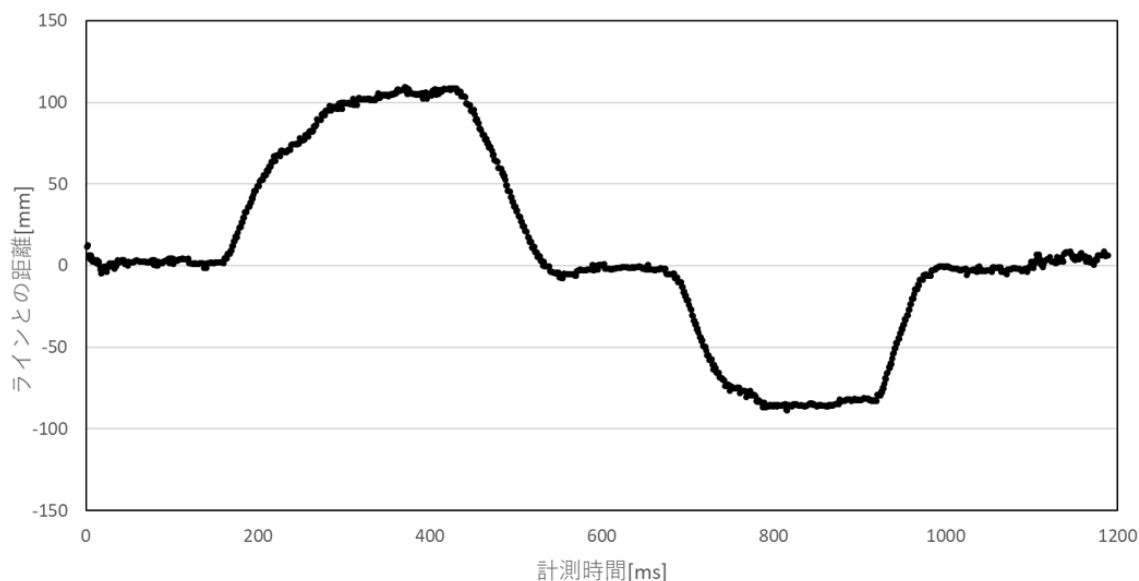


図 3.4 水平状態の機体とラインとの距離データ

図 3.4 より, ほぼ正確にラインとの距離を認識していることがわかる. 機体を手で持った状態で動かしているため, 多少の誤差はあるが, ライントレースする際には, 制御でカバーできる誤差の範囲であると考えられる.

次に, ライン上で機体を Roll 方向に約 ± 15 度回転させたとき, ラインとの距離が補正されているか確かめた計測データを図 3.7 に示す. また, Roll 方向に回転させた際の Φ の値のデータを図 3.6 に示す. その際の, カメラ座標系ではラインをどの座標で認識していたかを, カメラがラインを検知する 2 点の x 座標を図 3.5 に示す.

図 3.6 に示したような結果を得ることができた. Roll 方向に正の回転をさせたとき, カメラ座標系の x 座標の 2 点は正の数であり, Roll 方向に負の回転をさせたとき, カメラ座標系の x 座標の 2 点は負の数である. Roll の回転によってカメラから見えるラインの座標系は正常に反応していると

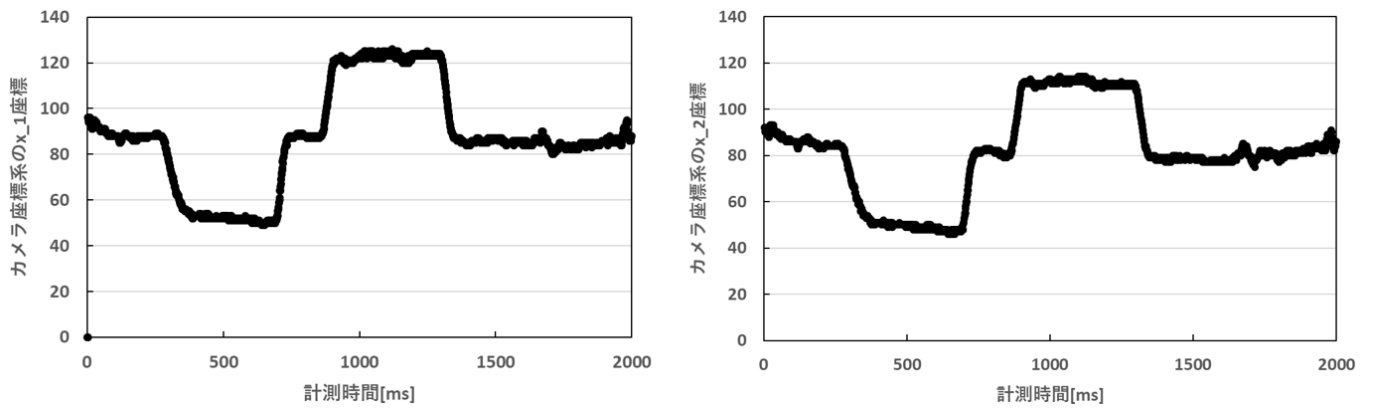


図 3.5 カメラ座標系の 2 点の x 座標

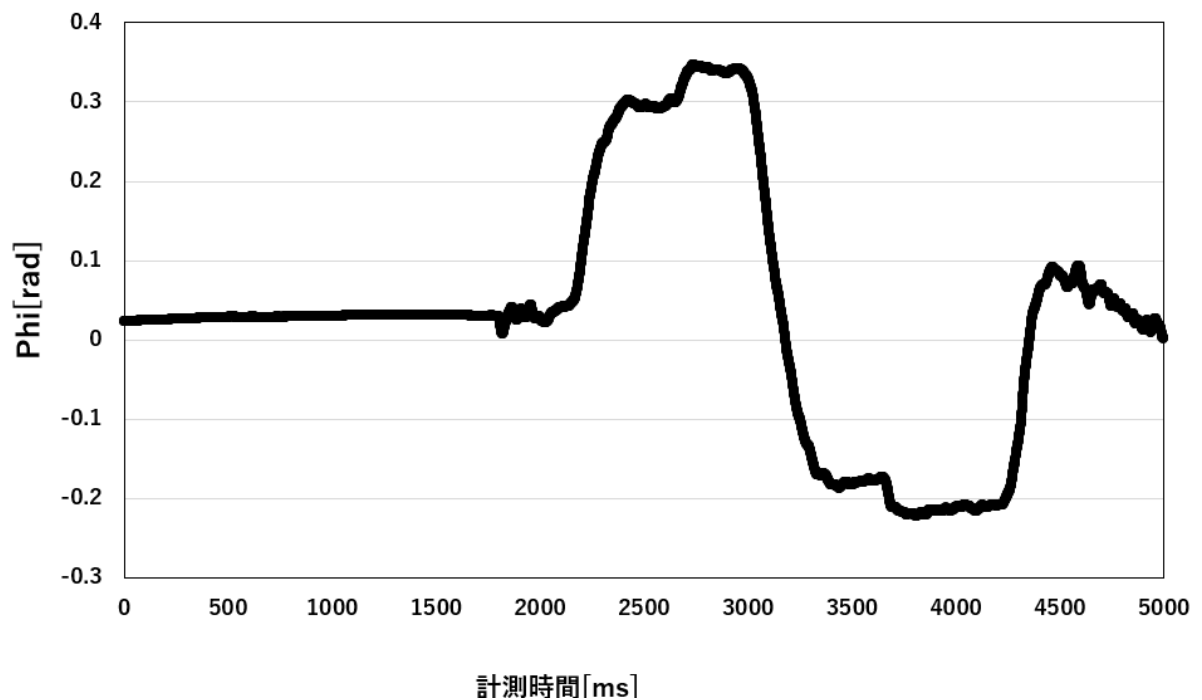


図 3.6 Roll 方向に回転させた機体の Phi の値のデータ

いえる。

3.3 実機での模擬飛行

座標変換により検出できたライン情報を使用してライントレースを想定した実機での模擬飛行を行った。模擬飛行を行う際に、毎秒 1m の速さで飛行することができる Pitch 方向の角度をつけ、実際のような Roll, Yaw 方向の回転を加え、約 30 秒間のデータを取得した。その際のラインとの距離のグラフを図 3.8 に示す。

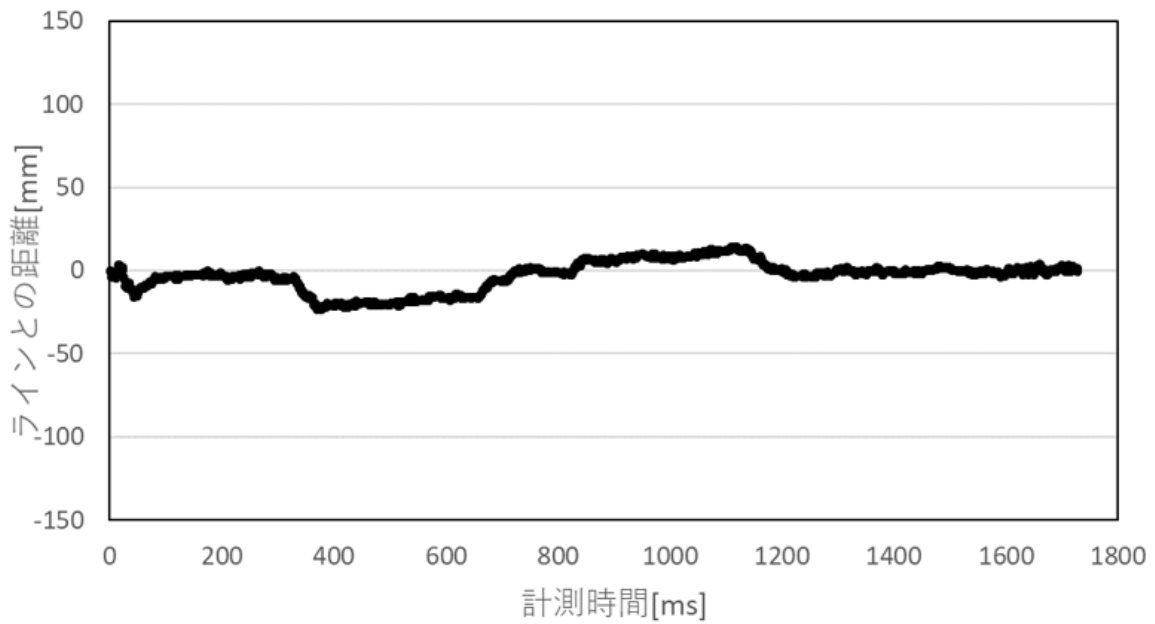


図 3.7 Roll 方向に回転させた機体とラインとの距離データ

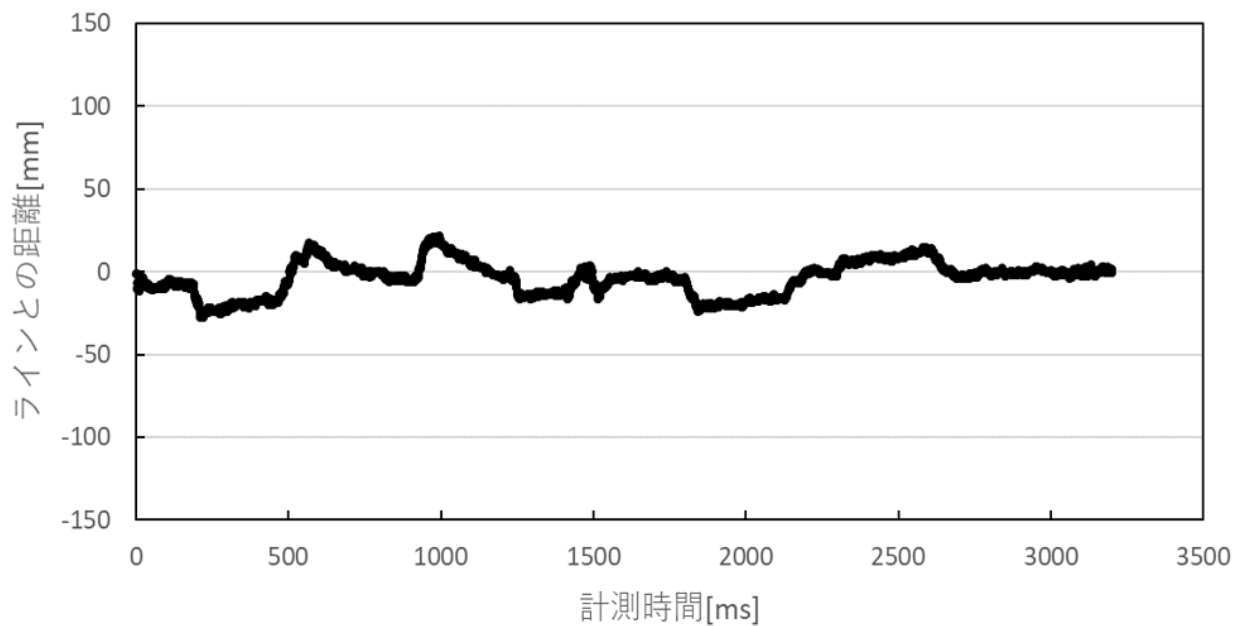


図 3.8 模擬飛行時のラインとの距離

3.4 ラインとの距離検出実験に対する考察

機体を Roll 方向 プラスマイナスの回転をさせたとき, 図 3.5 に示したように, カメラ画像上ではラインは離れているように見えている. しかし, カメラ画像座標系から慣性座標系への座標変換を用いることによって, 図 3.7 に示したように, ほぼライン上に機体が位置していると推定す

ることができている. 図 3.4 と図 3.7 を比較すると分かりやすいが, 図 3.4 の場合, カメラ画像上でラインとの距離が存在しているとなっており, 実際にラインとの距離は存在し正確に距離も推定できている. 図 3.7 の場合, カメラ画像上ではラインとの距離は存在しているように見えるが, 実際には機体はライン上に位置しているため, 機体の姿勢情報によりその姿勢分だけでキャンセルして, ライン上に位置していると推定できている. 機体を手で持って回転させているため, 多少の誤差は存在するが, ライントレースする際には制御のほうでカバーできる誤差の範囲であると考えられる.

第 4 章

Simulink による線形・非線形のライントレースシミュレーション

カメラでラインとの距離は出せるので,シミュレーションを用いてライントレースについて考える.本研究では,MATLAB の Simulink を用いて機体の方程式等を計算したものをブロックにし,PID のブロックや Scope などのブロックと繋げて,線形・非線形モデルを作成した.線形・非線形モデルのシミュレーションを作成することで,実機を壊す心配がなく,ゲイン調整の効率化を図ることができる.また,限りなく実機に近いパラメータでシミュレーションし,ゲイン調整することにより,実機でライントレースする際に,ゲイン調整がやりやすくなると考えた.マルチコプタの機体座標系を図 4.1 に示す.

4.1 線形シミュレーション

モータ + プロペラ系の回転の運動方程式 [4] を式 (4.1) に示す.

$$J\dot{\omega} + D\omega + C_Q\omega^2 + Q_f = Ki \quad (4.1)$$

回路の微分方程式 [4] を式 (4.2) に示す.

$$i = \frac{e - K\omega}{R} \quad (4.2)$$

式 (4.2) を式 (4.1) に代入し,その結果を線形化する.線形化した式は,式

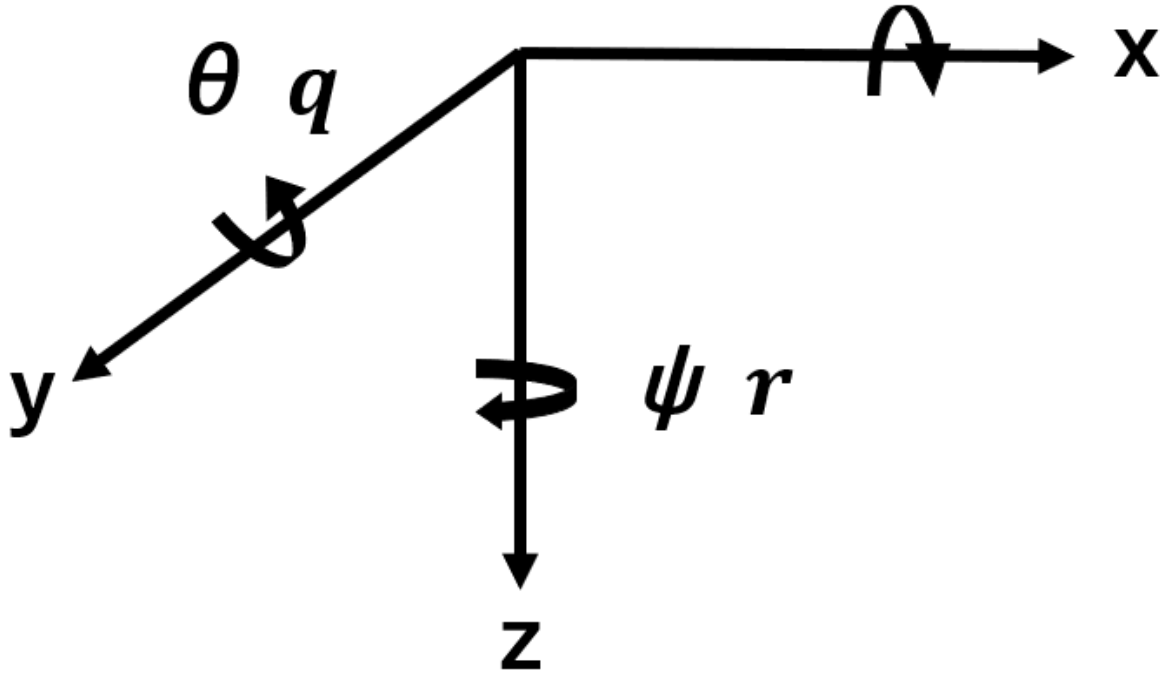


図 4.1 機体座標系のイメージ

(4.3) の様になる.

$$\omega = \frac{1}{s} \left\{ \frac{1}{J} \left(\frac{K}{R} e - \frac{K^2}{R} \omega - C_Q \omega^2 \right) \right\} \quad (4.3)$$

ここでの J は慣性モーメント [kgm^2], D は動粘性抵抗係数 [Nms/rad], R はコイル抵抗 [Ω], Q_f は摩擦トルク [Nm], e は印加電圧 [V], K はトルク定数, 逆起電圧定数 [Nm/A][Vs/rad] である.

マルチコプタを機体の運動方程式で表すと, 非線形なものであるため, ライントレース時のモデルをつくるためにはまず, マルチコプタを線形モデルで表す必要がある. 剛体としての機体の運動方程式の並進運動, 回転運動 [4] と考える. ライントレース時の機体の状態は, Pitch 方向に θ 分傾いているだけである. 機体は常に毎秒 1m で飛行するようにしたいので, 並進運動のつり合いの式 [4] 中の空力係数より, 機体の θ 角を求める必要がある. 空力係数を求める際にわかりやすくするため, ここで求める際だけ, 空力係数 C_u を X とおく. 並進運動のつり合いの式 [4] より,

$$m(\dot{u} + qw - rv) = -mg \sin \theta + X \quad (4.4)$$

式 (4.4) より 微小擾乱法を用いることで以下の変形が可能である.

$$\begin{aligned} X &= C_u \cdot U^2 \\ X_0 &= C_u \cdot u_0^2 \end{aligned} \quad (4.5)$$

また, つり合いの式の式より,

$$\begin{aligned} X_0 &= mg \sin \theta_0 \\ C_u &= \frac{mg \sin \theta_0}{u_0^2} \end{aligned} \quad (4.6)$$

上記の式と同様に C_u, C_w も求めることができる. また, 本研究において, マルチコプタの機体は前方向, 横方向はほぼ同じ形であるため, 空力特性も同じであると考え, $C_u = C_v$ とすることができる. 以上のことを踏まえ, u, v, w のそれぞれの空力係数を並進運動のつり合いの式 [4] に代入して, 並進運動の方程式を線形化された形にすると, 式 (4.7) のようになる.

$$\begin{aligned} \Delta u &= \frac{-w_0 \Delta q - g \Delta \theta \cos \theta_0}{s - \frac{2g \sin \theta_0}{u_0}} \\ \Delta v &= \frac{1}{s} (-u_0 \Delta r + w_0 \Delta p + \Delta \phi g \cos \theta_0) + \frac{2C_v v_0}{\Delta v} \\ \Delta w &= \frac{u_0 \Delta q - \Delta \theta g \sin \theta_0 - \frac{2C_T \omega_0}{m} (\Delta \omega_{FR} + \Delta \omega_{FL} + \Delta \omega_{RR} + \Delta \omega_{RL})}{s - \frac{2g \cos \theta_0}{w_0}} \end{aligned} \quad (4.7)$$

また, 常に毎秒 1m で飛行するようにしたいとき, 並進運動のつり合いの式より, $\theta = -6[deg]$ とすればよいわかった.

回転運動についての方程式を微小擾乱法を用いて線形化していくと, 式 (4.8) の様になる.

$$\begin{aligned} \Delta p &= \frac{1}{s} \cdot \frac{IC_t \omega_0}{I_x} (\Delta \omega_{FR} + \Delta \omega_{RR} - \Delta \omega_{FL} - \Delta \omega_{RL}) \\ \Delta q &= \frac{1}{s} \cdot \frac{IC_t \omega_0}{I_y} (\Delta \omega_{FR} + \Delta \omega_{RL} - \Delta \omega_{FL} - \Delta \omega_{RR}) \\ \Delta r &= \frac{1}{s} \cdot \frac{2C_Q \omega_0}{I_z} (\Delta \omega_{FR} + \Delta \omega_{RL} - \Delta \omega_{FL} - \Delta \omega_{RR}) \end{aligned} \quad (4.8)$$

式 (4.7), 式 (4.8) の I_x, I_y, I_z は慣性モーメント $[kgm^2]$, g は重力加速度 $[m/s^2]$, u, v, w は機体の速度 $[m/s]$, p, q, r は機体の角速度 $[rad/s]$, m は機体の質量, ϕ, θ, ψ はオイラー角 $[rad]$, C_u, C_v, C_w は空力係数である.

次にオイラー角の微分 [4] の式の線形化について考える. 線形化した式を式 (4.9) に示す.

$$\begin{aligned}\Delta\phi &= \frac{1}{s}(\Delta p + \tan\theta_0 \cdot \Delta r) \\ \Delta\theta &= \frac{1}{s}\Delta q \\ \Delta\psi &= \frac{1}{s} \cdot \frac{1}{\cos\theta_0}\Delta r\end{aligned}\tag{4.9}$$

速度の慣性空間への座標変換(オイラー角表現)[4]の式の線形化について考える. 線形化した式を式 (4.10) に示す.

$$\begin{aligned}\Delta X_e &= \frac{1}{s}\{(u_0 \cos\theta_0 + w_0 \sin\theta_0) + \Delta\theta(w_0 \cos\theta_0 - u_0 \sin\theta_0) + \Delta u \cos\theta_0 + \Delta w \sin\theta_0\} \\ \Delta Y_e &= \frac{1}{s}\{\Delta\psi(u_0 \cos\theta_0 + w_0 \sin\theta_0) + \Delta v - \Delta\phi w_0\} \\ \Delta Z_e &= \frac{1}{s}\{-\Delta\theta(u_0 \cos\theta_0 + w_0 \sin\theta_0) - \Delta u \sin\theta_0 + \Delta w \cos\theta_0\}\end{aligned}\tag{4.10}$$

これらの線形化した式を Simulink のブロックとしてつくっていき, 入力の順につなげることで機体に関する方程式を線形化したモデルをつくることができた. 作成したモデルの図 4.2 に示す.

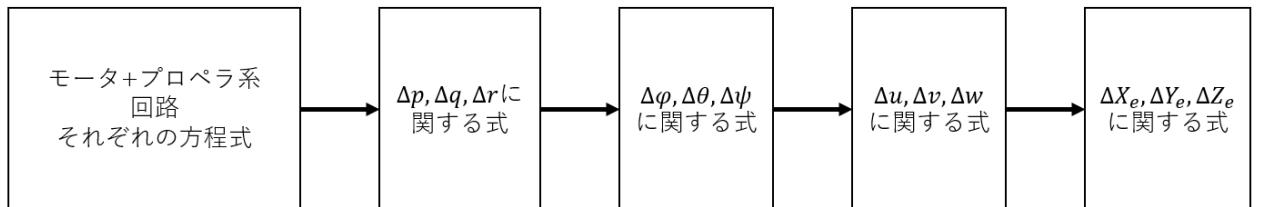


図 4.2 線形化した機体の方程式のモデル

図 4.2 を用いて, Simulink に機能として入っている PID のブロックを繋げていき, フィードバック制御の環境を作成した. 作成したフィードバック制御のモデルを図 4.3 に示す.

また, PID のブロックを繋げた詳細な制御系を図 4.4 に示す.

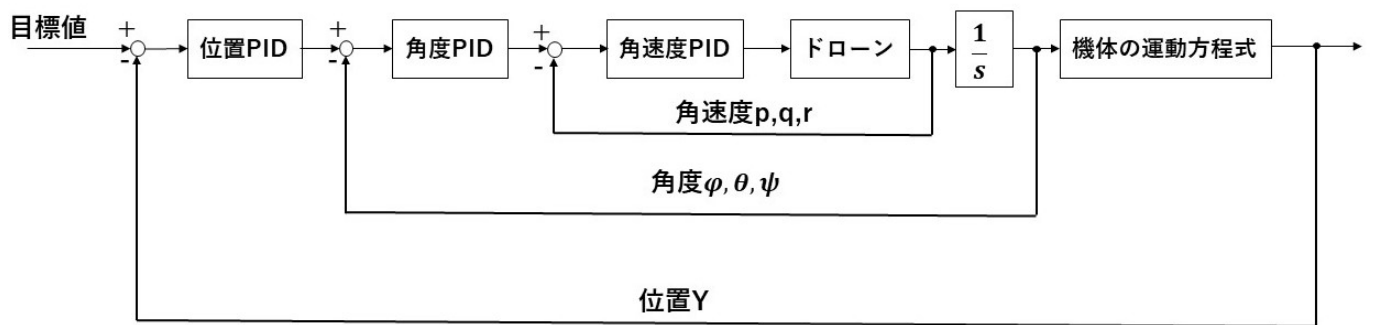


図 4.3 フィードバック制御のブロック線図

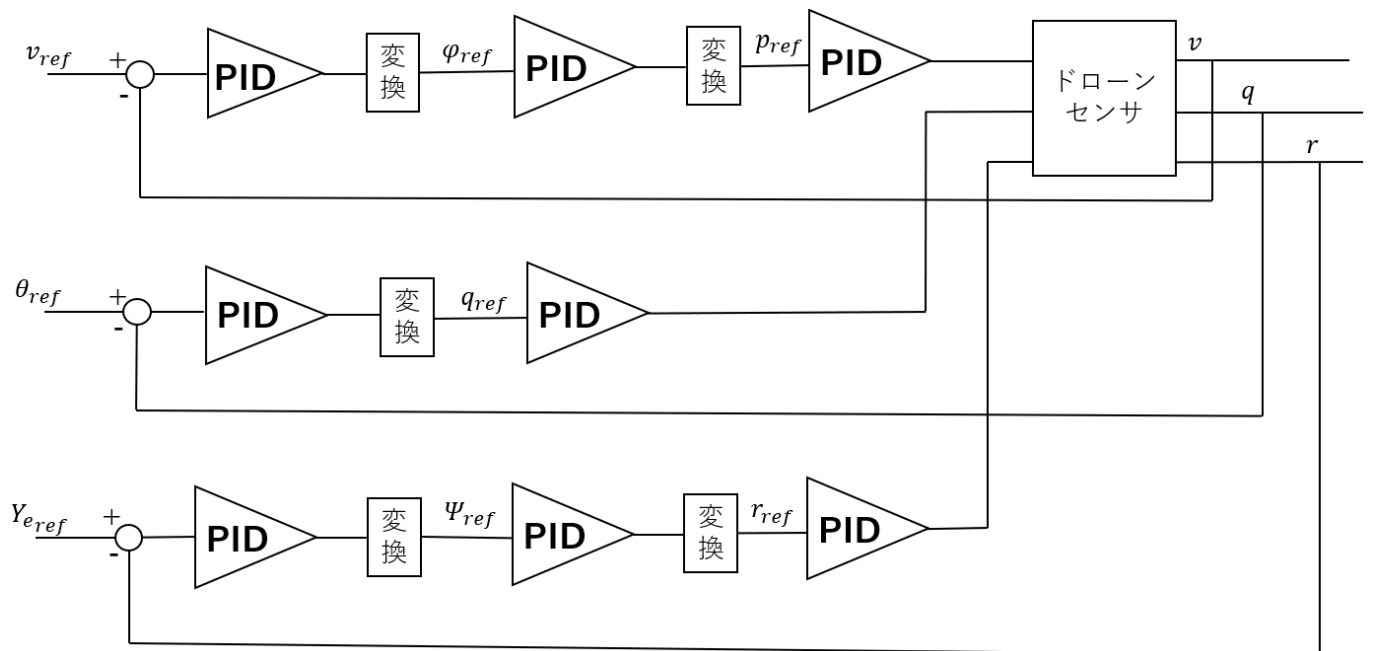


図 4.4 詳細な制御系

本研究では, マルチコプタの Y 方向がラインの中心との距離である. Y 方向の速度が 0 になるように機体に対する X 軸の回転角である Φ , その角速度の p を 0 に収束させる. また, ラインとの距離は機体に対する Z 軸の回転角である Ψ で調節する. また, 機体は常に毎秒 1m で飛行するように Y 軸の回転角である Θ の目標値を -6 度にして, フィードバック制御を行う. 図 4.3 のように Simulink に作成し, 制御したいものに対して PID 制御を行う. その際の波形結果を図 4.5 に示す. 図 4.5 に示した, 波形結果のラインとの距離である Y の初期値を 0.5[m] と設定し, 初期値から 0 に収束するまで安定し, 早い時間で収束するようなゲイン調整を行った.

図 4.5 はラインとの距離が 0 に収束する際のグラフであり, 左上のグラ

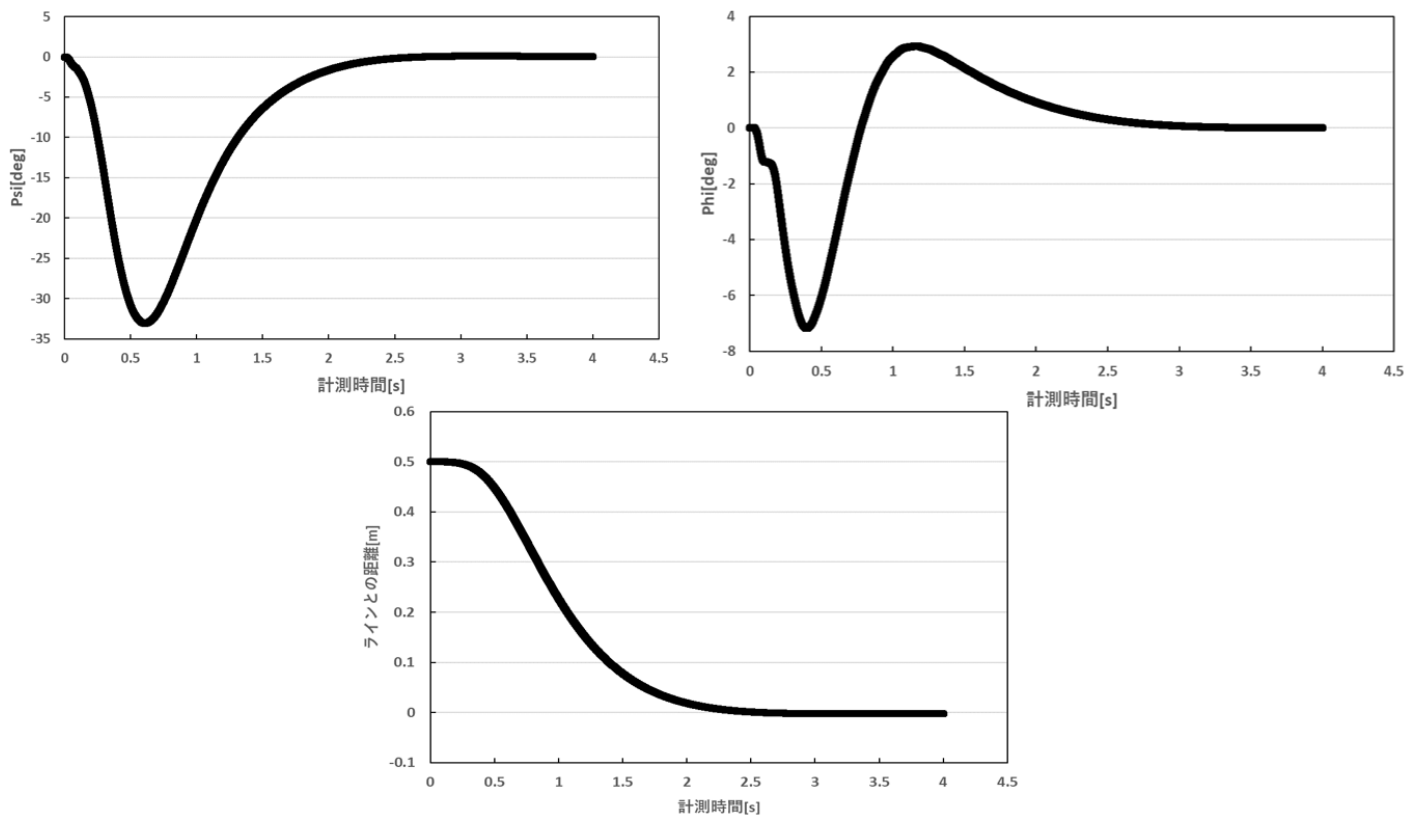


図 4.5 線形シミュレーション結果

フは Psi の変位, 右上のグラフは Phi の変位, 下のグラフは Y の変位 (ラインとの距離) を表している. それぞれグラフを見てわかるが, ラインとの距離は約 2 秒で 0 付近に収束し始め, 2.5 秒付近では 0 に収束していることがわかる. この線形シミュレーション結果において, ラインとの距離が 0.5[m] 離れても, 約 2.5 秒でラインとの距離を 0 に収めることができ, マルチコプタによるライントレースは不可能ではないことがわかる.

4.2 非線形シミュレーション

これまでは, 線形シミュレーションのことについて述べていたが, 実際に制御するマルチコプタの機体は, 非線形なものであり, 線形シミュレーションがうまくいったことにより実際に実機でライントレースできるということは確実ではない. そのため, マルチコプタの機体の非線形モデルを線形モデルを作成したと同じく, MATLAB の Simulink を用いて作成していく.

線形モデルも作成したときと同じく, まずは, 機体の運動方程式について解いていく. 並進運動のつり合いの式 [4] より,

$$\begin{aligned} u &= \frac{1}{s}(-qw + rv - g \sin \theta - \frac{C_u \cdot u^2}{m}) \\ v &= \frac{1}{s}(-ru + pw + g \cos \theta \sin \phi - \frac{C_v \cdot v^2}{m}) \\ w &= \frac{1}{s}\{-pv + qu + g \cos \theta \cos \phi + \frac{C_w \cdot w^2}{m} - C_T(\omega_{FL}^2 + \omega_{RL}^2 + \omega_{FR}^2 + \omega_{RR}^2)\} \end{aligned} \quad (4.11)$$

式 (4.11) のようになる. 次に回転運動のつり合いの式 [4] より,

$$\begin{aligned} p &= \frac{1}{s}\{-\frac{I_z - I_y}{I_x}qr + \frac{lC_T}{2I_y}(\omega_{FR}^2 + \omega_{FL}^2 - \omega_{FR}^2 - \omega_{RR}^2)\} \\ q &= \frac{1}{s}\{-\frac{I_x - I_z}{I_x}rp + \frac{lC_T}{2I_y}(\omega_{FR}^2 + \omega_{FL}^2 - \omega_{RR}^2 - \omega_{RL}^2)\} \\ r &= \frac{1}{s}\{-\frac{I_y - I_x}{I_z}pq + \frac{C_Q}{I_z}(\omega_{FR}^2 + \omega_{RL}^2 - \omega_{RR}^2) + \frac{K}{I_z}(\frac{e_{FR} - K\omega_{FR}}{R} + \frac{e_{FL} - K\omega_{FL}}{R} + \frac{e_{RR} - K\omega_{RR}}{R} + \frac{e_{RL} - K\omega_{RR}}{R})\} \end{aligned} \quad (4.12)$$

式 (4.12) のようになる. 次に線形モデルを作成した際には, オイラー角の微分を線形化したが, 今回は非線形モデルを作成するため, クォータニオンを用いて姿勢の角度の計算を行う. クォータニオンを用いる理由として, 2つの回転軸が同じ向きになってしまったときに, 回転の自由度が1つ落ちてしまう, ジンバルロックという現象が起きてしまうためである. また, メモリ的にもそこそこ優しく, 計算時間もはやい, 計算時に数値誤差が生じにくいなどたくさんのメリットがある. したがって, クォータニオンの微分の式 [4] より,

$$\begin{aligned} q_0 &= \frac{1}{s}\{0.5(-pq_1 - qq_2 - rq_3)\} \\ q_1 &= \frac{1}{s}\{0.5(pq_0 + rq_2 - qq_3)\} \\ q_2 &= \frac{1}{s}\{0.5(qq_0 - rq_1 + pq_3)\} \\ q_3 &= \frac{1}{s}\{0.5(rq_0 + qq_1 - pq_2)\} \end{aligned} \quad (4.13)$$

式 (4.13) のようになる. 次に速度の慣性空間への座標変換(クォータニオン表現)の式 [4] より,

$$\begin{aligned}
X_e &= \frac{1}{s} \{ (q_0^2 + q_1^2 - q_2^2 - q_3^2)u + 2(q_1q_2 - q_0q_3)v + 2(q_1q_3 + q_0q_2)w \} \\
Y_e &= \frac{1}{s} \{ 2(q_1q_2 + q_0q_3)u + (q_0^2 - q_1^2 + q_2^2 - q_3^2)v + 2(q_2q_3 - q_0q_1)w \} \\
Z_e &= \frac{1}{s} \{ 2(q_1q_3 - q_0q_2)u + 2(q_2q_3 + q_0q_1)v + (q_0^2 - q_1^2 + q_2^2 - q_3^2)w \}
\end{aligned} \tag{4.14}$$

式 (4.14) のようになる. しかし, 前述したように今回作成するモデルは, 非線形モデルであり, オイラー角表現ではジンバルロックの問題を解決することができないので, 式 (4.11) について, オイラー角表現からクォータニオン表現に変形する.

$$\begin{aligned}
u &= \frac{1}{s} \{ (-qw + rv + 2g(q_1q_3 - q_0q_2) - \frac{C_u \cdot u^2}{m}) \} \\
v &= \frac{1}{s} \{ (-ru + pw + 2g(q_2q_3 + q_0q_1) - \frac{C_v \cdot v^2}{m}) \} \\
w &= \frac{1}{s} \{ -pv + qu + g(q_0^2 - q_1^2 - q_2^2 + q_3^2) + \frac{C_w \cdot w^2}{m} - C_T(\omega_{FL}^2 + \omega_{RL}^2 + \omega_{FR}^2 + \omega_{RR}^2) \}
\end{aligned} \tag{4.15}$$

式 (4.15) のようにクォータニオン表現に変形することができた. しかし, 問題点として, クォータニオン表現が与えられたとき, 具体的な回転や姿勢をイメージしにくいということである. その問題解決のため, クォータニオンからオイラー角への変換, クォータニオンから回転行列への変換の2つのことを押さえることにした.[5]

今後説明する際に, 簡単にわかりやすく, かつ, 説明しやすくするため, 行列式を式 (4.16) のように定義する.

$$M = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix} \tag{4.16}$$

慣性座標系 → 機体座標系の座標変換行列 (方向余弦行列: DCM とオイラー角) は,

$$M = \begin{bmatrix} \cos\theta\cos\psi & \cos\theta\sin\psi & -\sin\theta \\ \sin\phi\sin\theta\cos\psi - \cos\phi\sin\psi & \sin\phi\sin\theta\sin\psi + \cos\phi\cos\psi & \sin\phi\cos\theta \\ \cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi & \cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi & \cos\phi\cos\theta \end{bmatrix} \tag{4.17}$$

式 (4.17) のようになる. また, 慣性座標系 → 機体座標系の座標変換行列 (方向余弦行列とクォータニオン) は,

$$M = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 + q_0q_3) & 2(q_1q_3 - q_0q_2) \\ 2(q_1q_2 - q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 + q_0q_1) \\ 2(q_1q_3 + q_0q_2) & 2(q_2q_3 - q_0q_1) & q_0^2 - q_1^2 - q_2^2 - q_3^2 \end{bmatrix} \quad (4.18)$$

式 (4.18) のようになる. 方向余弦行列からオイラー角に変換することができる. 変換の式を式 (4.21) に示す.

$$\phi = \tan^{-1} M_{23}/M_{33} \quad (4.19)$$

$$\theta = \tan^{-1} -M_{13}/\sqrt{M_{23}^2 + M_{33}^2} \quad (4.20)$$

$$\psi = \tan^{-1} M_{12}/M_{11} \quad (4.21)$$

式 (??) は, 式 (4.18), 式 (4.17) のそれぞれの行と列に対応しているため, オイラー角の検算やクォータニオンからオイラー角の変換に使用することができる.

Simulink で機体の運動方程式の計算のブロックを作成する場合, 線形モデルでは u_0 などのように, 初期値が入る変数の部分があったが, 非線形モデルでは初期値の入る変数がないため, 積分器のブロックに初期値を入れておく必要がある. 並進運動のつり合いの式では, 線形モデルで求めた u_0, v_0, w_0 の値を使用する. 回転運動のつり合いの式より, ω_0 を値を使用する. オイラー角の初期値設定はライントレースする際の姿勢を, 方向余弦行列で考えてから求める. まず, 座標系の整理をする. 座標系を図 4.6 に示す.

次に, ライントレース時の機体の姿勢を初期値に設定するため, ライントレース時の姿勢を図 4.7 に示す.

図 4.6 のどの軸まわりにも回転していない状態の機体座標系の回転行列は式 (4.22) のようになる.

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (4.22)$$

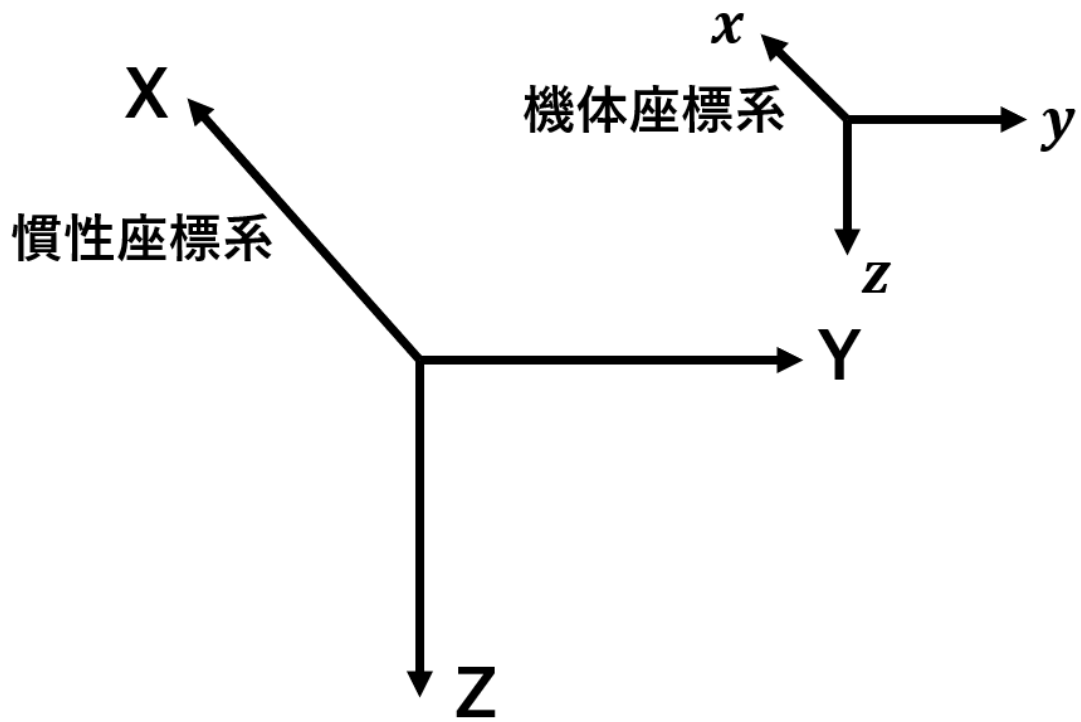


図 4.6 座標設定

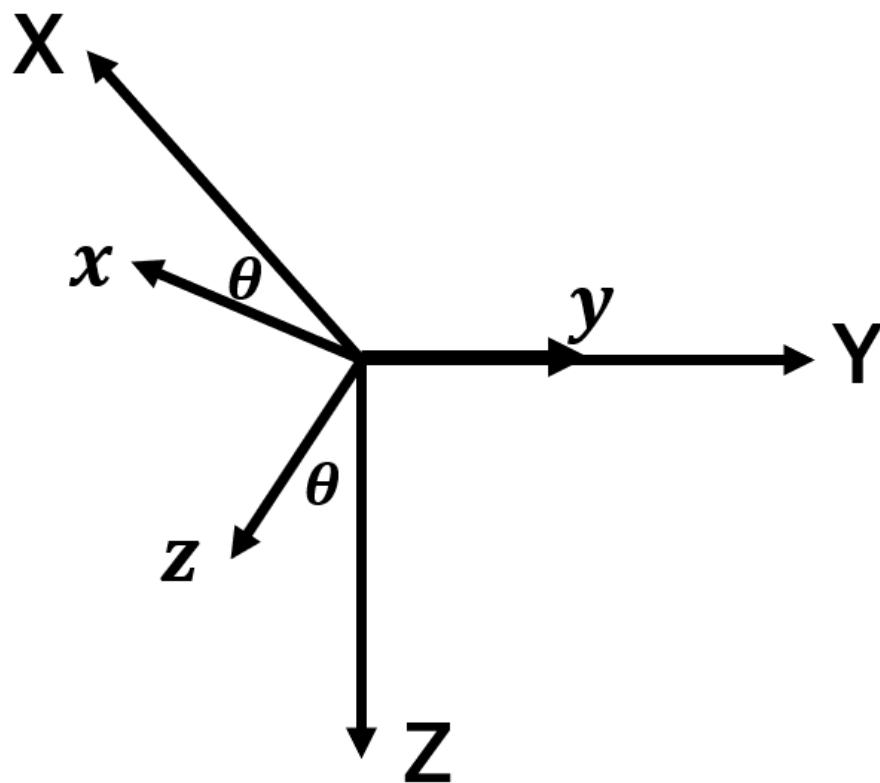


図 4.7 座標設定

図.4.7 の状態のときは Y 軸まわりの回転である. そのときの機体座標系の回転行列で表すと, 式 (4.23) のようになる.

$$M = \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \quad (4.23)$$

図 4.7 の状態の式 (4.23) により, オイラー角表現の回転行列で表すことができた. 式 (4.17) と式 (??) を用いることにより, θ だけが存在する場合を考え, その他の変数は 0 と扱うことによって検算することができる. 検算の結果, 図 4.7 の状態で式 (4.23) は正しいことがわかる.

ライントレース時の姿勢をオイラー角表現の回転行列を用いて表すことができたため, オイラー角表現からクォータニオン表現に変形することにより初期値を求めることができる. その際, 式 (4.24) を用いる必要がある.

$$\begin{aligned} q_0 &= 0.5\sqrt{1 + M_{11} + M_{22} + M_{33}} \\ q_1 &= (M_{23} - M_{32})/4q_0 \\ q_2 &= (M_{31} - M_{13})/4q_0 \\ q_3 &= (M_{12} - M_{21})/4q_0 \end{aligned} \quad (4.24)$$

式 (4.24) に示した行列の行と列の位置を表す M_{11} のような変数は, オイラー角表現の方向余弦行列である式 (4.17) の行列の変数と対応しており, ライントレース時の姿勢を表している 4.23 の行列の位置にも対応している. そのため, 式 (4.24) の行列の位置を表す文字に式 (4.23) の行列の位置を表す文字と対応する部分を代入する. すると, $q_0 = 0.7071, q_1 = 0.3516, q_2 = q_3 = 0$ と計算結果が出る. したがって, クォータニオンの初期値をライントレース時の姿勢から方向余弦行列を用いることにより, 求めることができた.

速度の慣性空間への座標変換は, 式 (4.14) に示したように, 初期値が設定済みのクォータニオンや並進運動のつり合いの式が入力となっているため, 初期値の設定は行わう必要がない. しかし, クォータニオンについてまだ考える必要がある. クォータニオンの定義において, 回転軸は, 通常ノルムは 1 に統一して考える. したがって, 回転軸は方向だけが重要であり, 大きさには意味がないので, ノルムを 1 に統一する必要がある

[6]. したがって, 式 (4.13) によって出てきた q_0, q_1, q_2, q_3 を $\sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2}$ で割る必要がある. このような正規化する Simulink のブロックを図 4.8 に示す.

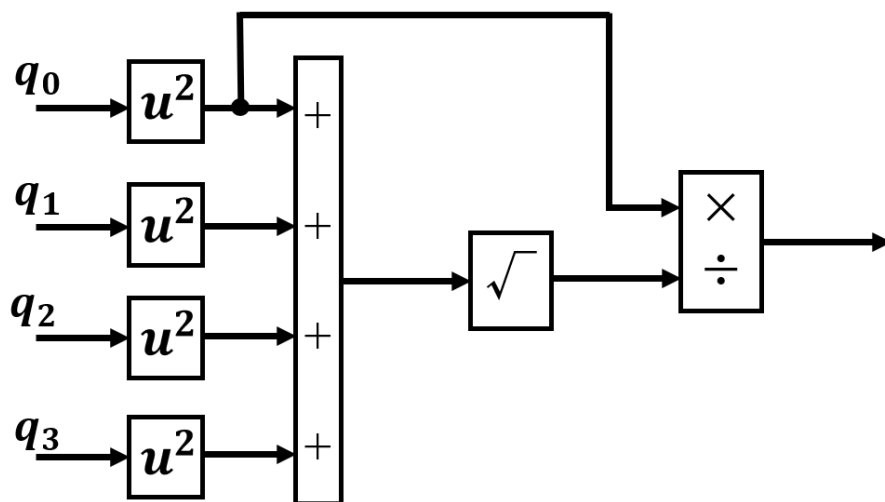


図 4.8 Simulink での正規化のブロック

図 4.8 は, q_0 についての正規化のブロックである. このブロックを q_1, q_2, q_3 のそれぞれが入力のものを作成する. 初期値にも正規化する必要がある. 式 (4.24) と式 (4.23) から求めたクォータニオンの初期値 $q_0 = 0.7071, q_1 = 0.3516, q_2 = q_3 = 0$ を正規化すると, $q_0 = 0.8954, q_1 = 0.4452, q_2 = q_3 = 0$ となった. このように非線形モデルをつくる前段階での準備をすることができたので, 線形モデルのように非線形モデルを作成する. 作成した非線形モデルの Simulink のブロックを図 4.9 に示す.

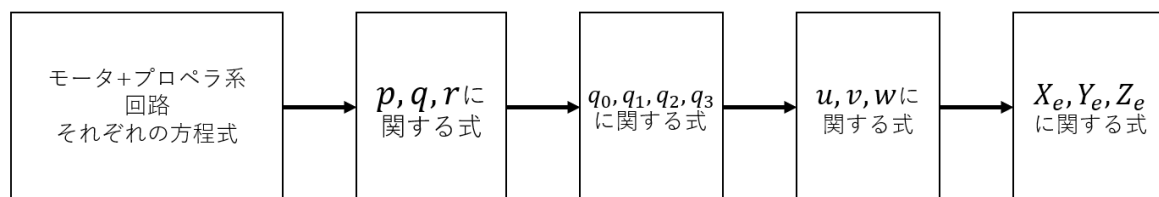


図 4.9 Simulink の非線形モデル

線形モデルと同じように, 図 4.9 を PID ブロックとつなげ, 図 4.4 と図 4.3 のようにフィードバック制御をかけて, シミュレーションを行う. 本来マルチコプタは非線形なものであるため, 線形モデルが非線形モデルでもうまくいくかを確認する. そのため, 線形シミュレーションと同じ PID のゲインでシミュレーションを実行する. 実行したシミュレーションの波形結果を図 4.10 に示す. 図 4.10 に示した, 波形結果のラインとの距

離である Y の初期値を $0.5[m]$ と設定し, これは, はじめ機体とラインとの距離が $0.5[m]$ 離れているということを示している.

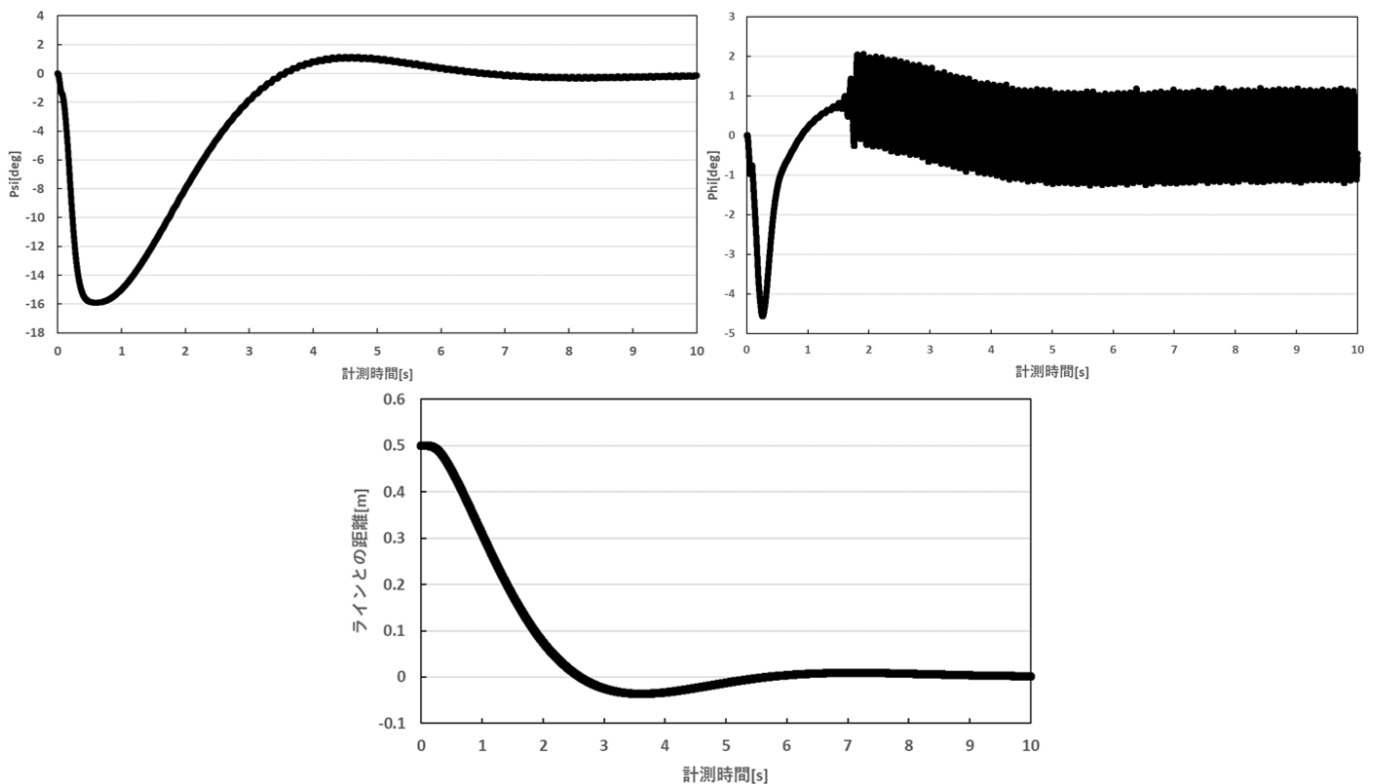


図 4.10 非線形モデルのシミュレーション結果

図 (4.10) より, ラインとの距離が 0 に収束する際のグラフであり, 左上のグラフは Psi の変位, 右上のグラフは Phi の変位, 下のグラフは Y の変位 (ラインとの距離) を表している. それぞれグラフを見てわかるが, ラインとの距離は約 3 秒で 0 付近に収束し始め, 4 秒付近では 0 に収束していることがわかる.

4.3 線形シミュレーションと非線形シミュレーションの結果による考察

線形モデルでのシミュレーション結果も非線形モデルでのシミュレーション結果もどちらもラインとの距離が 0 に収束し始めるのにかかった時間は 2 秒ぐらいであった. これらの結果により, マルチコプタによるライントレースは可能であると考えることができる. また, マルチコプタは本来非線形なものである. そのため, 線形モデルを作成し, シ

シミュレーションしたときのPIDゲインを非線形モデルにも同じPIDゲインを入れてシミュレーションをしても同じような結果を得ることができるかを確かめた。結果として、図(4.5)、図(4.10)に示したように、同じようなシミュレーション結果を得ることができ、線形モデルは正常に機能しているといえる。この結果から、機体がラインから0.5[m]離れると、約2秒から3秒の間に収束すると分かる。ライントレースする上では、問題ないと考えられる。また、本研究では空力特性の部分を厳密に調べなかったため、風洞実験などを行って空力係数を出せるとよりリアルな結果を得ることができると考えた。また、本研究で使用したマルチコプタのフレームや配線、その他の取り付け部品など自作で開発したため、細かく分解することが可能であり、慣性モーメントなどのシミュレーションで使用するパラメータを計算することが可能であった。そのため、ほぼ使用する実機と同じ環境下でシミュレーションができたため、実機でライントレースする際には、今回のシミュレーションで使用したPIDゲインを参考にしたいと考えている。

第 5 章

おわりに

5.1 結論

機体の固定されたカメラのカメラ画像のライン情報により、ラインと機体を同じ慣性座標空間上に置くことで、マルチコプタの姿勢によってラインとの距離が変化してしまう問題を解決できた。したがって、ラインと機体との距離を正確に算出することが可能になった。ライントレースできる範囲でのラインとの距離の誤差まで小さくすることができた。また、マルチコプタの線形・非線形モデルのライントレースシミュレーションを MATLAB の Simulink を用いて作成したことにより、マルチコプタによるライントレースの実現性を示すことができた。

5.2 今後の課題

ラインと機体との距離は算出することができたので、実機によりライントレースさせることが今後の課題である。40[m] ほどの直線のライントレースを成功させたい。また、機体の前進方向 (X 軸方向) の速度の制御についても考えたい。前進方向の速度制御をできることによって、着陸地点を検知して、その上でホバリングといったように制御しやすくなると考えている。

参考文献

- [1] 春原久徳,『2023 年度、日本国内におけるドローンビジネスの現状と今後の展望』,ver2023 年
- [2] 小森脩平,岡田佳樹,山田樹,『自律飛行を目的としたドローンの飛行制御』,(<https://www.st.nanzan-u.ac.jp/info/gr-thesis/2020/nakashima/pdf/17sc088.pdf>202 年 1 月 27 日 アクセス)
- [3] 全日本学生室内飛行ロボットコンテスト,(<http://indoor-flight.com/2023> 年 1 月 27 日 アクセス)
- [4] 伊藤恒平,『マルチコプタの運動と制御の基礎のきそ』,(https://www.docswell.com/s/Kouhei_Ito/KDVNVK-2022-06-15-193343 2023 年 11 月 30 日 アクセス)
- [5] 柳原正明,『飛行シミュレーションアルゴリズム』,宇宙航空研究開発機構研究開発機構 ver2021 年 2 月 19 日
- [6] 株式会社 NTT データ数理システム,『クォータニオン (Quaternion) を総整理! 三次元物体の回転と姿勢を鮮やかに扱う』,(<https://qiita.com/drken/items/0639cf34cce14e8d58a5> 2024 年 1 月 27 日 アクセス)

謝辞

本論文は筆者が金沢工業大学工学部ロボティクス学科に在籍中の研究成果をまとめたものである。同学科教授伊藤先生には指導教官として本研究の実施の機会を与えて戴き、その遂行にあたって終始、ご指導を戴いた。ここに深謝の意を表する。同学科伊藤研究室の各位には研究遂行にあたり日頃より有益なご討論ご助言を戴いた。ここに感謝の意を表する。